

POSIX Threads Producer-Consumer Synchronization Using Semaphores

This multithreaded C program demonstrates thread communication and resource coordination using POSIX threads, semaphores, and a mutex.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS_TO_PRODUCE 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty; // Counts empty slots
sem_t full; // Counts filled slots
pthread_mutex_t mutex;

/* Producer Thread */
void *producer(void *arg) {
    for (int item = 1; item <= ITEMS_TO_PRODUCE; item++) {

        sem_wait(&empty); // Wait for empty slot
        pthread_mutex_lock(&mutex); // Enter critical section

        buffer[in] = item;
        printf("Producer produced item %d at index %d\\n",
            item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // Leave critical section
        sem_post(&full); // Signal item available

        sleep(1);
    }

    return NULL;
}

/* Consumer Thread */
void *consumer(void *arg) {
    for (int i = 0; i < ITEMS_TO_PRODUCE; i++) {

        sem_wait(&full); // Wait for available item
        pthread_mutex_lock(&mutex); // Enter critical section

        int item = buffer[out];

        printf("Consumer consumed item %d from index %d\\n",
```

```

        item, out);

    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex); // Leave critical section
    sem_post(&empty);             // Signal empty slot

    sleep(2);
}

return NULL;
}

int main() {
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL,
                  producer, NULL);

    pthread_create(&consumer_thread, NULL,
                  consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nAll threads completed successfully.\n");

    return 0;
}

```

/*

Compilation:

```
gcc -pthread producer_consumer.c -o producer_consumer
```

Example Output:

```

Producer produced item 1 at index 0
Consumer consumed item 1 from index 0
Producer produced item 2 at index 1
Producer produced item 3 at index 2
Consumer consumed item 2 from index 1
Producer produced item 4 at index 3

```

...

All threads completed successfully.

*/

Explanation

Synchronization Mechanism: 1. Semaphore 'empty' tracks the number of available slots in the buffer. If the buffer is full, the producer blocks at `sem_wait(&empty)`. 2. Semaphore 'full' tracks the number of produced items. If the buffer is empty, the consumer blocks at `sem_wait(&full)`. 3. A mutex protects the shared buffer and index variables (in and out). Only one thread can modify the shared data at a time. 4. `sem_post()` wakes waiting threads when resources become available.

Why Synchronization Is Needed: Without semaphores and mutexes, the producer and consumer could access the buffer simultaneously, causing race conditions, overwritten data, lost items, or invalid reads. Synchronization guarantees safe shared-memory access and ensures threads execute in the correct order when resources are available.